TOCTOU - OilRig

Incident analysis report - Workstation compromise and SYSTEM escalation (CVE-2024-30088)

Affected asset	Windows 10 workstation of user mashle (192.168.1.27)
Malicious files	`CLEAR_EDDY.exe` (initial payload), `TOCTOU.exe` (privilege-escalation exploit), `toctou[1].bat` (persistence and cleanup script), `winhlp32.exe` (masqueraded copy)
Vulnerability exploited	CVE-2024-30088 (Windows kernel time-of-check to time-of-use race condition, local privilege escalation)
Evidence analyzed	Memory image `mem.dmp` and the process, file-system, registry and network artifacts extracted from it
Tools used	MemProcFS, Volatility 3, Timeline Explorer, DB Browser for SQLite, Registry Explorer, Notepad++, DCode, PowerShell, VirusTotal
Overall result	Confirmed compromise: SYSTEM-level access, rogue administrator account, two persistence mechanisms, event-log clearing and an established HTTPS connection to an external address

1. Executive Summary

A researcher using the account mashle was looking for fluid-dynamics modeling tools and downloaded an archive named `CLEAR_EDDY.zip` from an anonymous file-sharing service. The program inside it was a malicious executable. I examined a memory image of the affected workstation to work out what the program did once it ran and how far the intruder got.

The image shows a complete intrusion. The user launched `CLEAR_EDDY.exe` on 22 February 2025 at 12:43:13 UTC. About half an hour later a second-stage tool, `TOCTOU.exe`, was pulled down with the built-in `certutil.exe` utility and used to abuse **CVE-2024-30088**, which gave the attacker a **SYSTEM-level** process at 13:12:43 UTC. The attacker then created a local administrator account named **gu3st**, copied the malware into System32 as `winhlp32.exe`, wired that copy into a PowerShell profile for persistence, and cleared the Windows event logs at 13:26:09 UTC. When the memory was captured, `winhlp32.exe` held an established HTTPS connection to **162.159.140.245**. The workstation must be treated as fully compromised.

2. Scope and Evidence

This report covers a single Windows workstation. The only evidence is a memory image, `mem.dmp`, captured with `DumpIt.exe` at about 15:42 UTC on 22 February 2025. I mounted and processed the image with MemProcFS, which exposes process listings, a file-system and process timeline, recovered files and registry hives as ordinary files and CSV exports, and I used Volatility 3 for the UserAssist registry records. The analysis tools were Notepad++ and Timeline Explorer for the text and CSV output, DB Browser for SQLite for the Edge history database, Registry Explorer for the SAM hive, DCode for timestamp conversion and PowerShell for file hashing. All times in this report are UTC.

The case is labelled OilRig, but the artifacts alone do not support attributing the activity to a named group, so this report sticks to observed behavior.

3. Background: CVE-2024-30088 and the Artifacts Used

The vulnerability. CVE-2024-30088 is a Windows kernel elevation-of-privilege flaw classified as a time-of-check to time-of-use race condition (CWE-367) with a CVSS score of 7.0 (High). The flaw sits in the kernel implementation of `NtQueryInformationToken`, which does not take proper locking when it works on an object. It cannot be exploited remotely: the attacker must already be running code on the machine. By winning the race between the check and the use, that code can end up with SYSTEM privileges. That is why the exploit binary in this case was named TOCTOU.

What to look for. A local privilege-escalation exploit leaves a recognizable trail: a tool arriving on disk shortly before a process suddenly appears with a SYSTEM token in an unexpected place in the process tree, followed by actions only an administrator could perform, such as creating accounts, writing into System32 and clearing logs.

Artifacts. The investigation relies on several Windows artifacts. **UserAssist** registry values record programs the user launched through the shell, with a run count and the last run time. **Prefetch** files record that an executable ran and are normally written about ten seconds after the launch. **Edge history** is a SQLite database whose timestamps are Chromium-style values: microseconds since 1 January 1601 UTC. The **SAM hive** holds local accounts and group membership, and **INetCache** holds files fetched through the Windows internet stack, which includes downloads made with certutil.

4. Investigation Phases

Phase 1 - Triage of the Process List

I started with the process listings that MemProcFS builds: the tree view (`proc.txt`, Figure 1) and the same processes ordered by creation time (`proc-time.txt`, Figure 2). The tree describes a normal interactive session. `winlogon.exe` (PID 560) and `userinit.exe` start `explorer.exe` for the user mashle, Microsoft Edge is running with its usual swarm of child processes, and `DumpIt.exe` appears last as a child of Explorer, which is the memory acquisition itself.

Two entries do not belong. `CLEAR_EDDY.exe` (PID 5948) is a direct child of `winlogon.exe` and runs as SYSTEM, from 13:12:43 to 13:29:15 on 22 February. `winhlp32.exe` (PID 6364) also runs as SYSTEM and started at 13:26:08; its parent (PID 6872) is no longer in the list. In the time-ordered view, a cluster of shell and service processes starts at around 12:42, which is the user activity during which the download from the previous evening was launched. These two processes became the leads for the rest of the investigation.

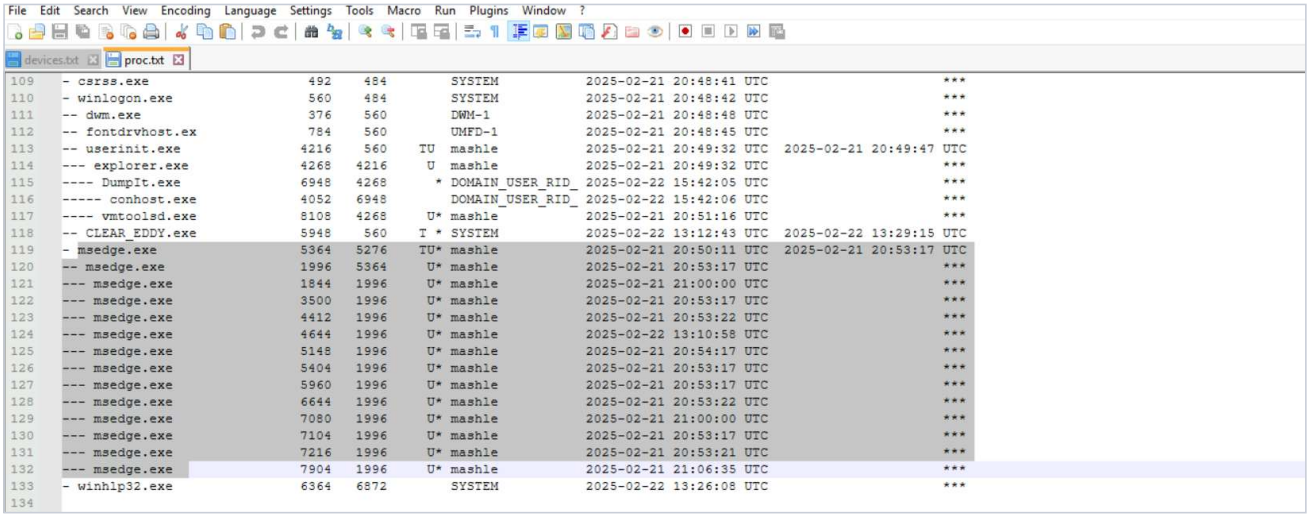
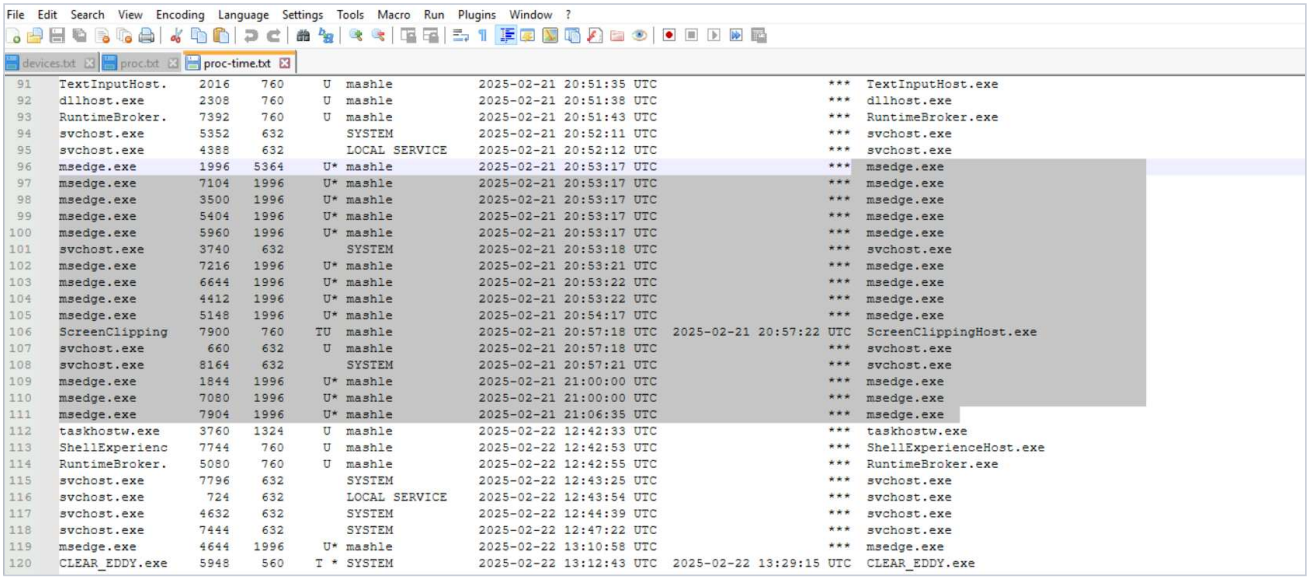


Figure 1. Process tree (proc.txt). CLEAR_EDDY.exe (PID 5948) runs under winlogon.exe (PID 560) as SYSTEM; winhlp32.exe (PID 6364) also runs as SYSTEM; Edge processes are highlighted.



91	TextInputHost.	2016	760	U	mashle	2025-02-21 20:51:35 UTC		***	TextInputHost.exe
92	dllhost.exe	2308	760	U	mashle	2025-02-21 20:51:38 UTC		***	dllhost.exe
93	RuntimeBroker.	7392	760	U	mashle	2025-02-21 20:51:43 UTC		***	RuntimeBroker.exe
94	svchost.exe	5352	632		SYSTEM	2025-02-21 20:52:11 UTC		***	svchost.exe
95	svchost.exe	4388	632		LOCAL SERVICE	2025-02-21 20:52:12 UTC		***	svchost.exe
96	msedge.exe	1996	5364	U*	mashle	2025-02-21 20:53:17 UTC		***	msedge.exe
97	msedge.exe	7104	1996	U*	mashle	2025-02-21 20:53:17 UTC		***	msedge.exe
98	msedge.exe	3500	1996	U*	mashle	2025-02-21 20:53:17 UTC		***	msedge.exe
99	msedge.exe	5404	1996	U*	mashle	2025-02-21 20:53:17 UTC		***	msedge.exe
100	msedge.exe	5960	1996	U*	mashle	2025-02-21 20:53:17 UTC		***	msedge.exe
101	svchost.exe	3740	632		SYSTEM	2025-02-21 20:53:18 UTC		***	svchost.exe
102	msedge.exe	7216	1996	U*	mashle	2025-02-21 20:53:21 UTC		***	msedge.exe
103	msedge.exe	6644	1996	U*	mashle	2025-02-21 20:53:22 UTC		***	msedge.exe
104	msedge.exe	4412	1996	U*	mashle	2025-02-21 20:53:22 UTC		***	msedge.exe
105	msedge.exe	5148	1996	U*	mashle	2025-02-21 20:54:17 UTC		***	msedge.exe
106	ScreenClipping	7900	760	TU	mashle	2025-02-21 20:57:18 UTC	2025-02-21 20:57:22 UTC	***	ScreenClippingHost.exe
107	svchost.exe	660	632	U	mashle	2025-02-21 20:57:18 UTC		***	svchost.exe
108	svchost.exe	8164	632		SYSTEM	2025-02-21 20:57:21 UTC		***	svchost.exe
109	msedge.exe	1844	1996	U*	mashle	2025-02-21 21:00:00 UTC		***	msedge.exe
110	msedge.exe	7080	1996	U*	mashle	2025-02-21 21:00:00 UTC		***	msedge.exe
111	msedge.exe	7904	1996	U*	mashle	2025-02-21 21:06:35 UTC		***	msedge.exe
112	taskhostw.exe	3760	1324	U	mashle	2025-02-22 12:42:33 UTC		***	taskhostw.exe
113	ShellExperienc	7744	760	U	mashle	2025-02-22 12:42:53 UTC		***	ShellExperienceHost.exe
114	RuntimeBroker.	5080	760	U	mashle	2025-02-22 12:42:55 UTC		***	RuntimeBroker.exe
115	svchost.exe	7796	632		SYSTEM	2025-02-22 12:43:25 UTC		***	svchost.exe
116	svchost.exe	724	632		LOCAL SERVICE	2025-02-22 12:43:54 UTC		***	svchost.exe
117	svchost.exe	4632	632		SYSTEM	2025-02-22 12:44:39 UTC		***	svchost.exe
118	svchost.exe	7444	632		SYSTEM	2025-02-22 12:47:22 UTC		***	svchost.exe
119	msedge.exe	4644	1996	U*	mashle	2025-02-22 13:10:58 UTC		***	msedge.exe
120	CLEAR_EDDY.exe	5948	560	T *	SYSTEM	2025-02-22 13:12:43 UTC	2025-02-22 13:29:15 UTC	***	CLEAR_EDDY.exe

Figure 2. Process list ordered by creation time (proc-time.txt). The next-day activity begins at 12:42 and CLEAR_EDDY.exe appears at 13:12:43 with an exit time of 13:29:15.

Phase 2 - Identifying the Download

Because the user worked in Edge, I recovered the browser History database through MemProcFS and opened it in DB Browser for SQLite. The `downloads` table has two rows (Figure 3). The first is a reference document saved to the Desktop whose referrer is a publisher website, the kind of material a researcher would collect. The second is `C:\Users\mashle\Downloads\CLEAR_EDDY.zip`, which stands out immediately.

The referrer column of the second row shows that the archive came from `https://bashupload.com/prk5V/CLEAR_EDDY.zip` (Figure 4), an anonymous upload service rather than a software vendor. The row stores its start time as the Chromium value `13384645592735454`. Decoding it in DCode as Chromium time in microseconds (Figure 5) gives **21 February 2025, 21:06:32 UTC**. The recorded end time is about 3.3 seconds later, so this was a small, quick download. This is the start of the infection chain.

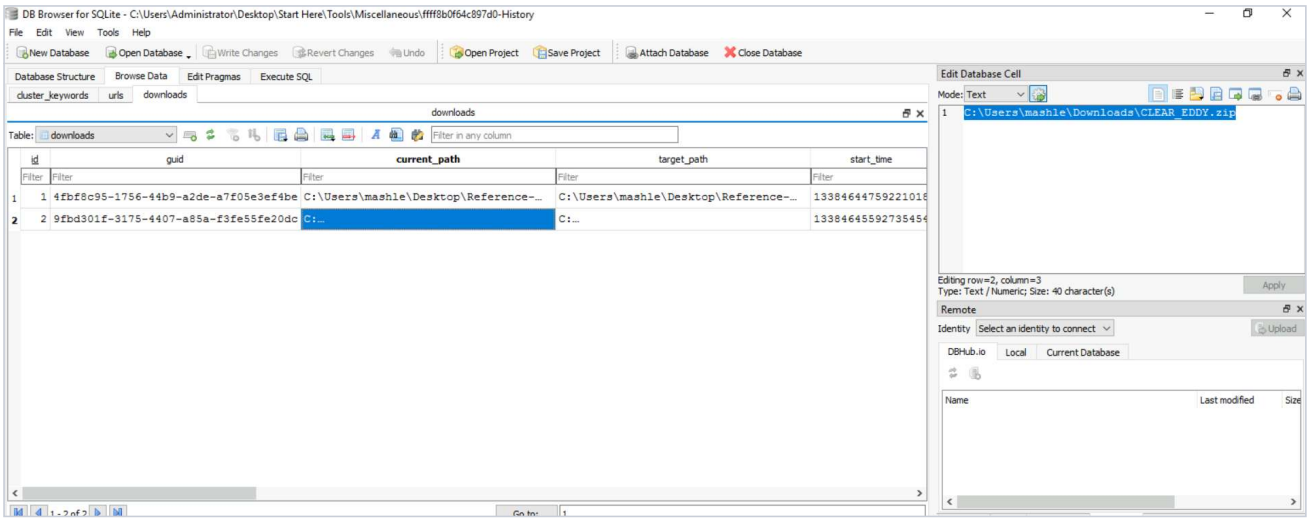


Figure 3. Edge History database, downloads table: the reference document and CLEAR_EDDY.zip saved to C:\Users\mashle\Downloads.

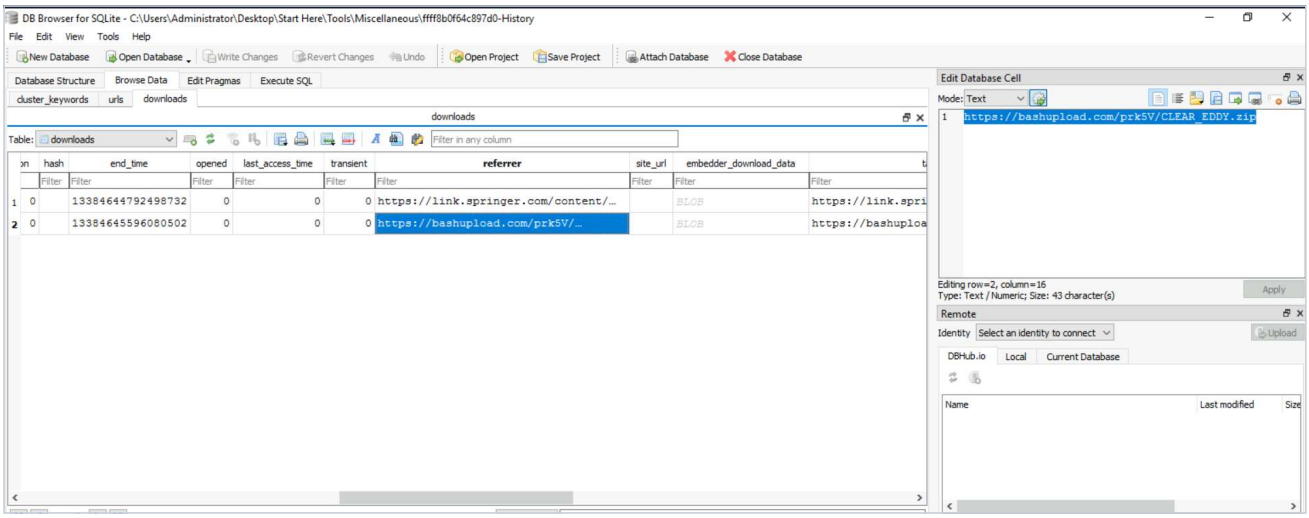


Figure 4. Referrer of the second download: bashupload.com/prk5V/CLEAR_EDDY.zip.

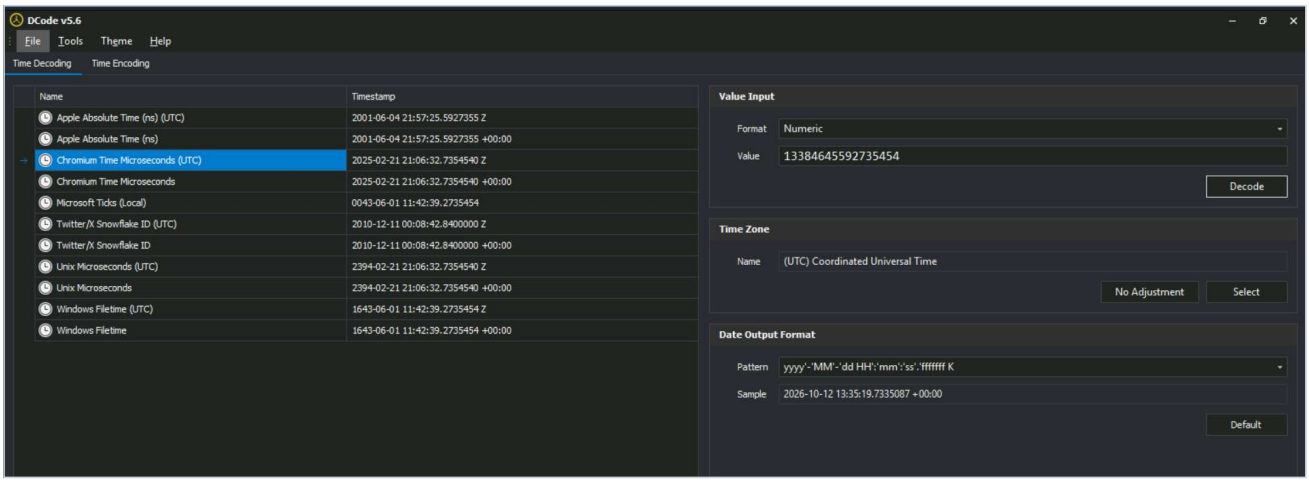


Figure 5. DCode decoding the Chromium timestamp 13384645592735454 to 2025-02-21 21:06:32 UTC.

Phase 3 - Execution of the Downloaded Binary

The verbose process listing stores each process's image path. Searching it for the word downloads returns exactly one hit: `\Device\HarddiskVolume3\Users\mashle\Downloads\CLEAR_EDDY.exe` (Figure 6). The program therefore ran straight out of the Downloads folder, which fits the archive having been unpacked by hand.

To find when the user first launched it, I exported the UserAssist records with Volatility 3 (`vol3 -f mem.dmp -r pretty windows.registry.userassist.UserAssist > userassist.txt`). The entry for `C:\Users\mashle\Downloads\CLEAR_EDDY.exe` shows a run count of 1 and a last-run time of **22 February 2025, 12:43:13 UTC** (Figure 7). That is roughly 15.5 hours after the download. Entries for DumpIt (the acquisition tool run from a VMware shared folder) and 7-Zip sit in the same list, and the 7-Zip entry would fit with the archive being opened by hand.

Searching the combined timeline for the file name (Figure 8) backs this up and adds a second event. A Prefetch file for `CLEAR_EDDY.EXE` is created at 12:43:23, ten seconds after the UserAssist time, which is the normal delay. Then a process-creation event for PID 5948 tagged as SYSTEM appears at 13:12:43, the Prefetch file is modified at 13:12:53, and the process is deleted at 13:29:15. In other words the program ran twice: once as the user at 12:43, and again as SYSTEM about thirty minutes later.

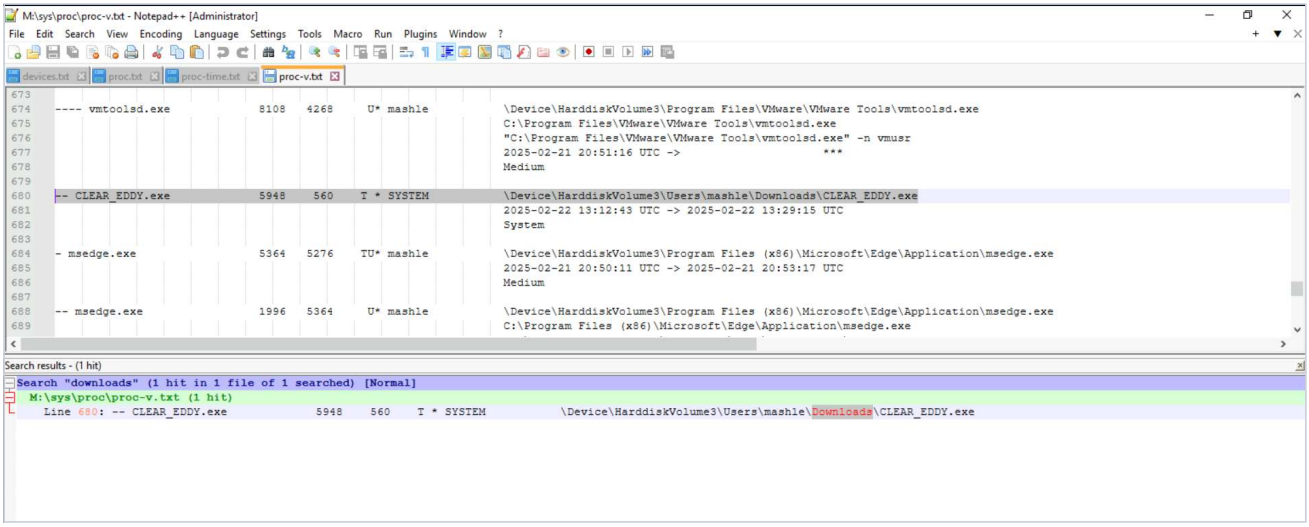


Figure 6. Verbose process list: the image path of CLEAR_EDDY.exe is the user's Downloads folder (search for "downloads" returns one hit).

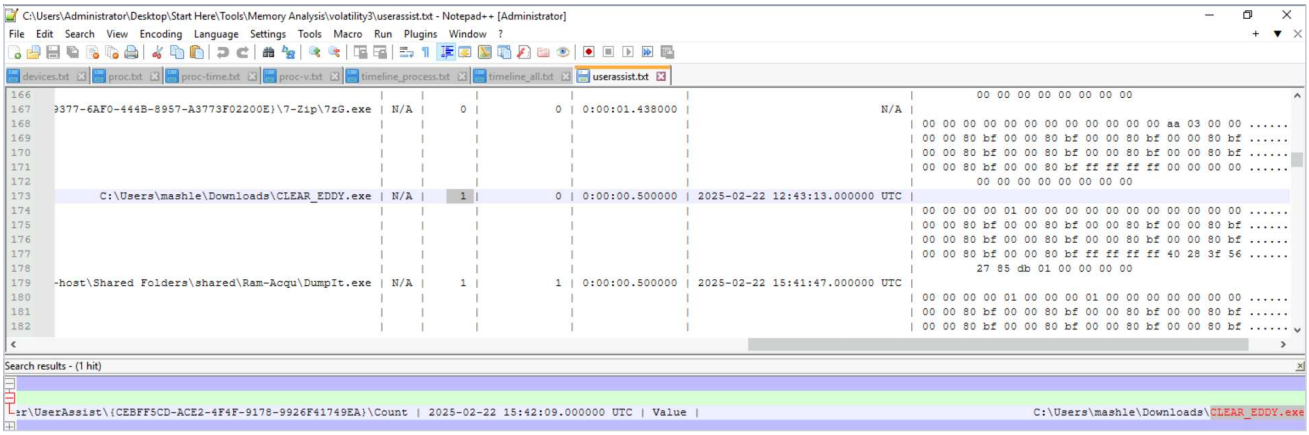


Figure 7. UserAssist output from Volatility 3: CLEAR_EDDY.exe, run count 1, last executed 2025-02-22 12:43:13 UTC.

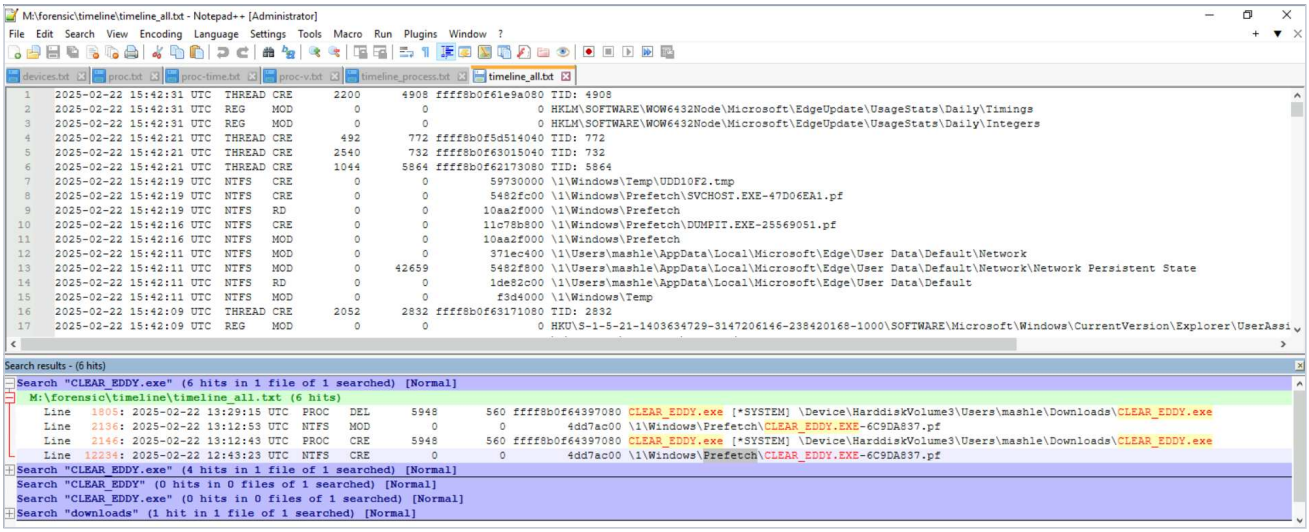


Figure 8. Combined timeline searched for CLEAR_EDDY.exe: Prefetch created at 12:43:23, SYSTEM process created at 13:12:43, Prefetch modified at 13:12:53, process deleted at 13:29:15.

Phase 4 - Second-Stage Retrieval with a Built-in Utility

To see what happened in the thirty minutes between the two runs, I filtered the file-system timeline in Timeline Explorer for Prefetch entries (Figure 9). The sequence is telling: `MSEDGE.EXE` at 13:10:23, `CERTUTIL.EXE` at 13:10:47, `TOCTOU.EXE` created at 13:12:43, `CLEAR_EDDY.EXE` modified at 13:12:53 and `WHOAMI.EXE` at 13:13:45. Certutil is a Windows certificate tool that can also download files from a URL, and it has no business running on a researcher's workstation. Carving strings from the memory image and searching them for certutil (`strings -el mem.dmp | grep certutil -C 10`) turned up a command line using it to fetch `TOCTOU.exe` from a GitHub-hosted URL. This is a living-off-the-land technique: the download is done by a signed Microsoft binary, so a block list of bad programs would not see it.

I then searched the file listing for batch files (Figure 10). Apart from VMware, Edge and Windows servicing files, one stands out: `toctou[1].bat` in the user's INetCache, created and modified at 13:10:47, the same second that the certutil Prefetch entry was updated. That suggests the script was fetched the same way as the exploit. Searching the file list for toctou (Figure 11) shows both cached files side by side: `TOCTOU[1].exe` and `toctou[1].bat`. The WHOAMI Prefetch entry a minute after the SYSTEM process appeared is consistent with the operator checking which account they had become.

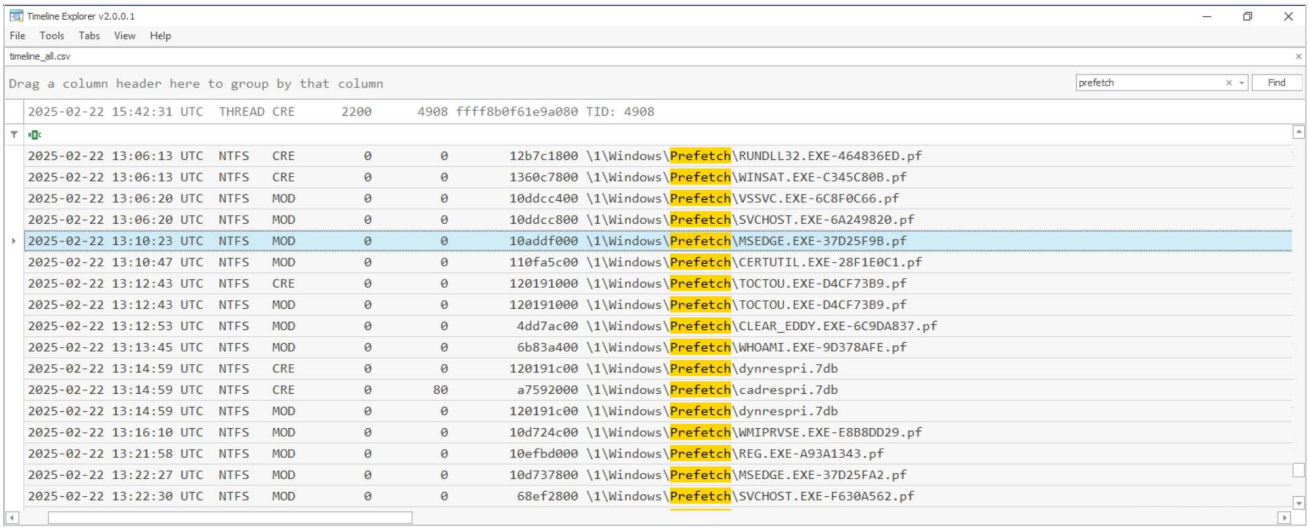


Figure 9. Prefetch entries in the file-system timeline: MSEDGE (13:10:23), CERTUTIL (13:10:47), TOCTOU and CLEAR_EDDY (13:12:43 and 13:12:53), WHOAMI (13:13:45).

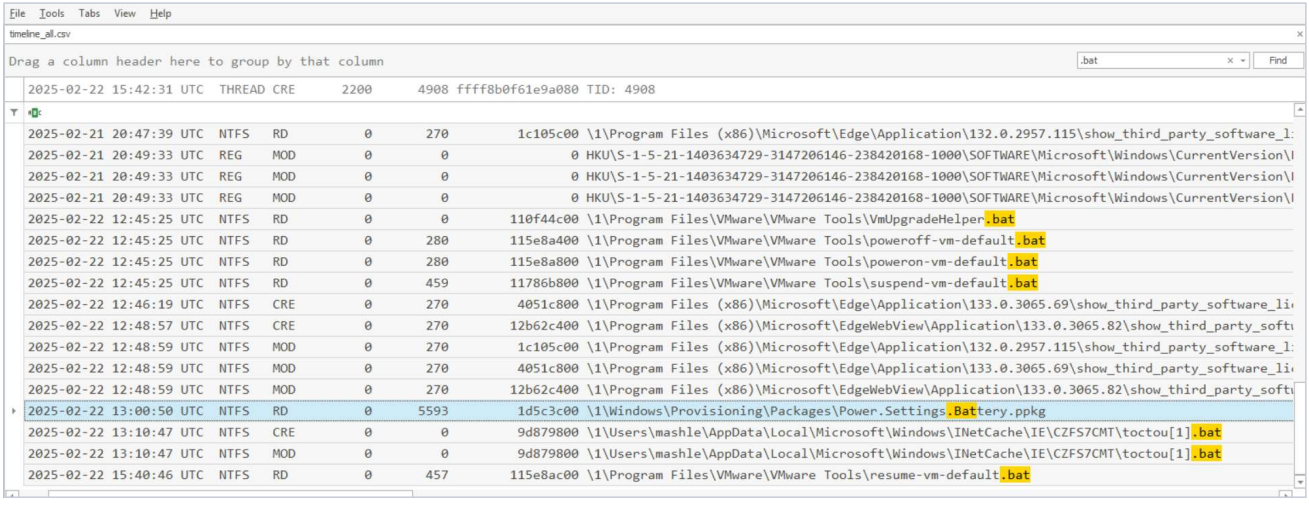


Figure 10. Timeline search for .bat files: toctou[1].bat is created in INetCache at 13:10:47 alongside legitimate VMware and Edge scripts.

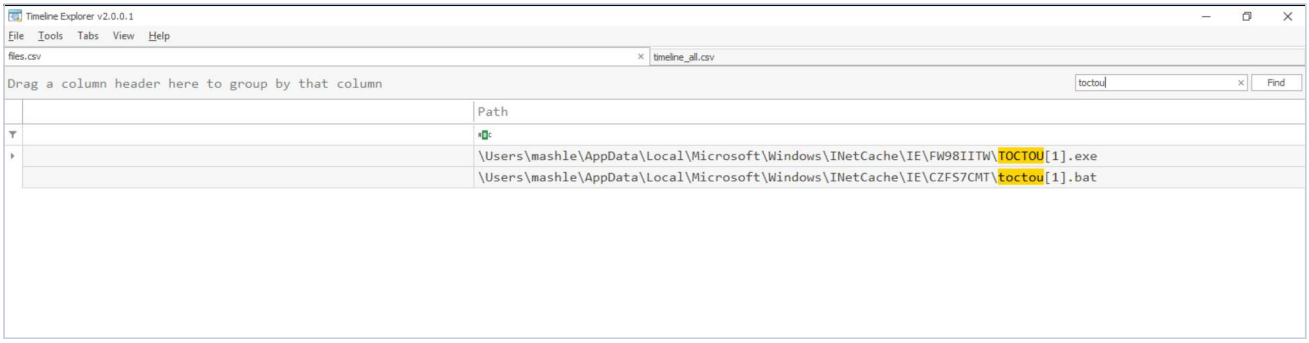


Figure 11. File listing searched for "toctou": TOCTOU[1].exe and toctou[1].bat, both in the user's INetCache.

Phase 5 - Analysis of the Cached Executable

I browsed to the recovered copy of the executable, which MemProcFS names with a kernel address prefix (Figure 12). It is a 1,024 KB application. Hashing it in PowerShell gives an MD5 of `72FA3477AE8D9EBCD3266352E2C4D2D5` (Figure 13) and a SHA-256 of `51c4e44fbe6764253887259923e8420ae70dffb3a19aed0d006567eecebbf3d9`.

A reputation lookup on the SHA-256 (Figure 14) shows 36 of 72 vendors flagging the file, with the popular threat label trojan.tedy/convagent and tags including `cve-2024-30088` and `exploit`. The vendor list (Figure 15) is the most useful part, because several detection names reference the vulnerability directly: Exploit:Win/CVE-2024-30088.A, W64/CVE_2024_30088.C!exploit and a variant of Win64/Exploit.CVE-2024-300... . The detection count will drift over time, but the naming is stable. This file is a local privilege-escalation exploit for CVE-2024-30088, and its name matches the race-condition class of the bug.

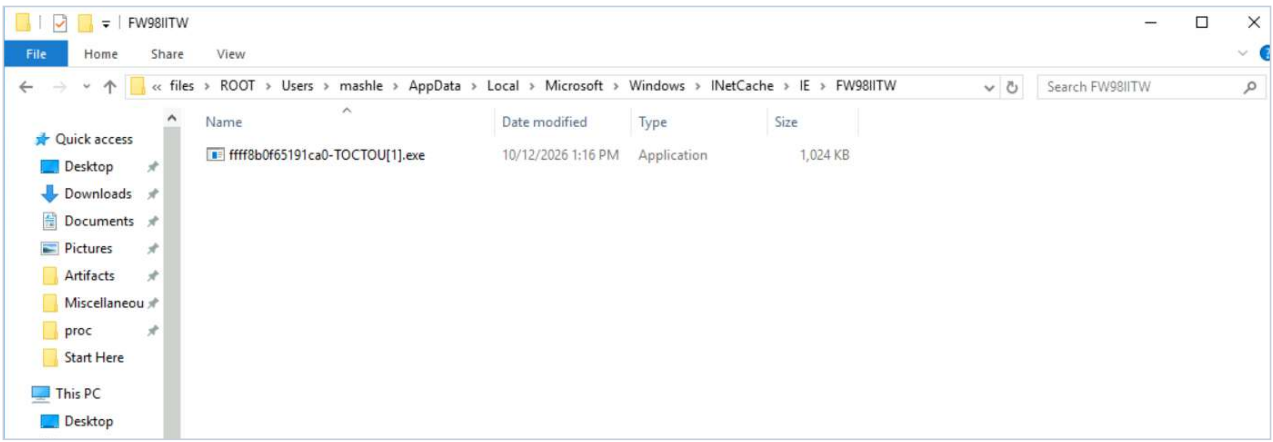


Figure 12. Recovered TOCTOU[1].exe (1,024 KB) in the extracted INetCache folder.

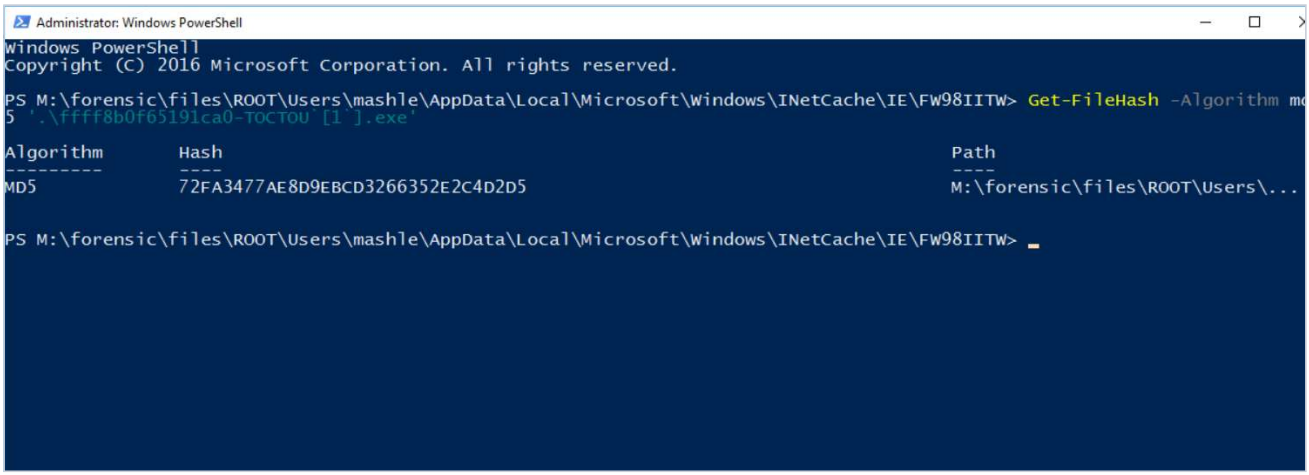


Figure 13. PowerShell Get-FileHash output for TOCTOU[1].exe (MD5).

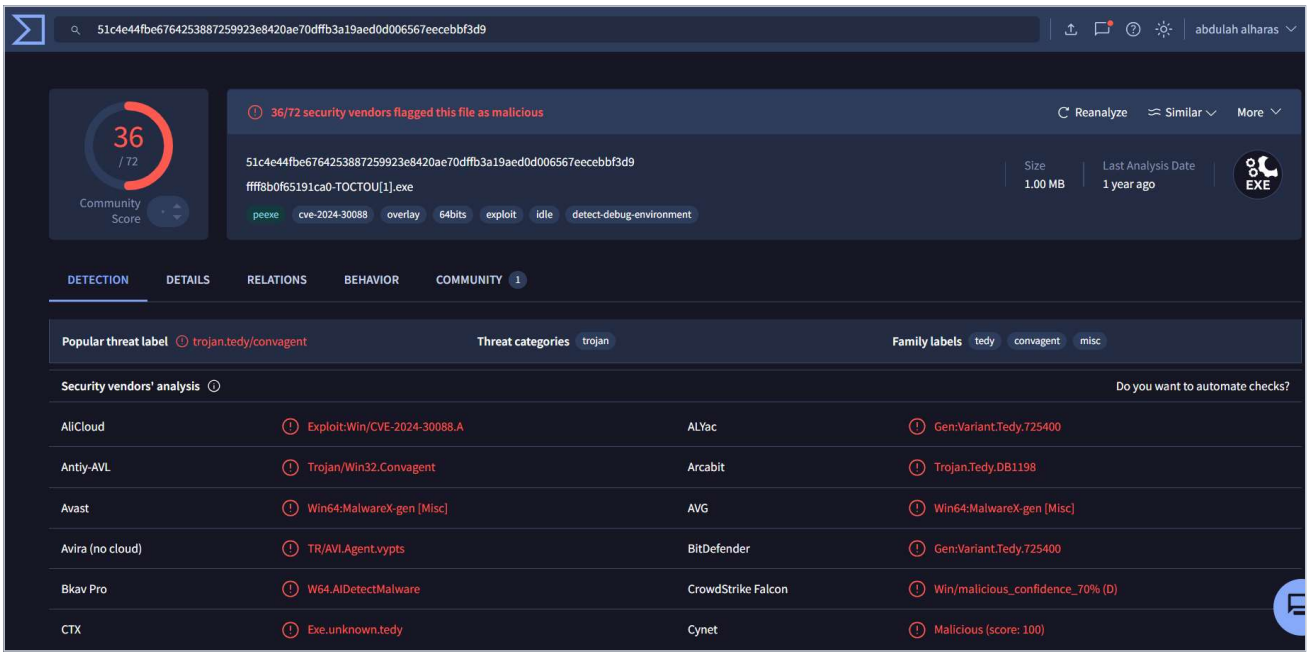


Figure 14. Reputation report for the SHA-256: 36 of 72 vendors flag it, tagged cve-2024-30088 and exploit.

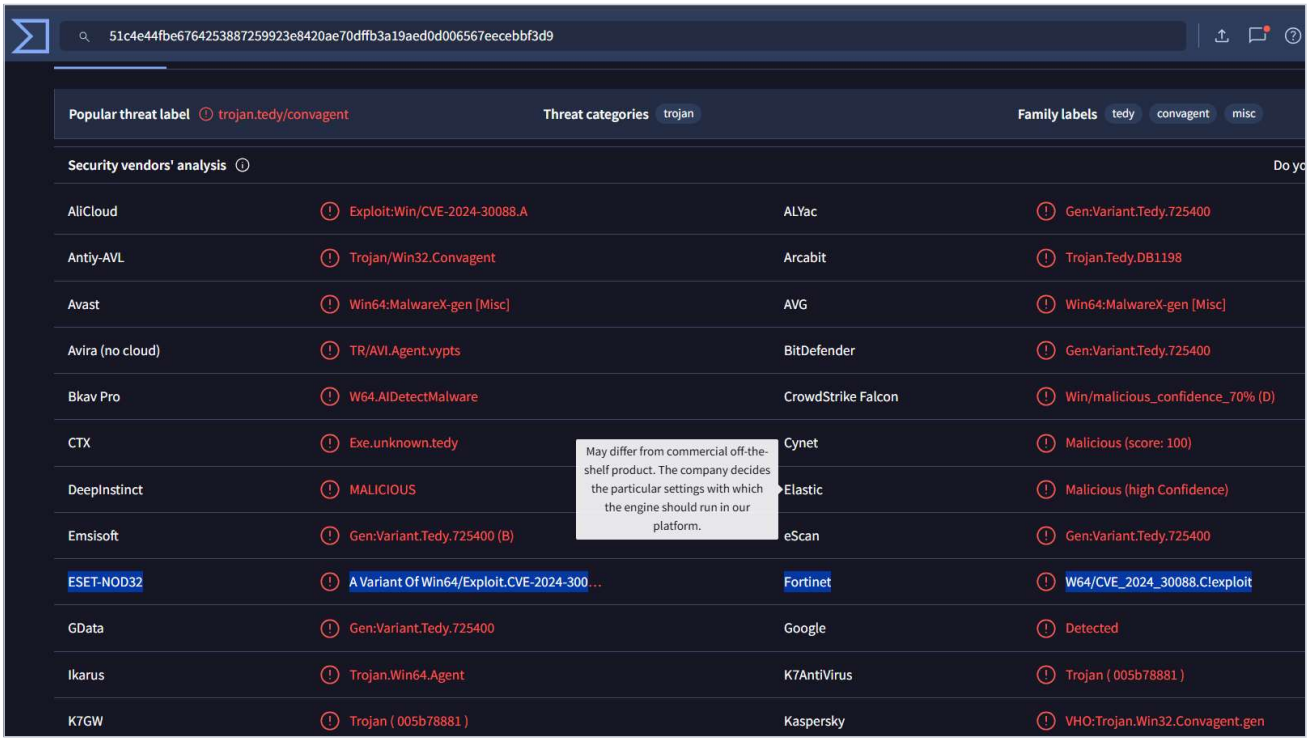


Figure 15. Vendor detections referencing CVE-2024-30088, including Fortinet and ESET.

Phase 6 - Privilege Escalation to SYSTEM

Back in the verbose process list, the search for CLEAR_EDDY returns a single entry (Figure 16): PID 5948, parent PID 560, user SYSTEM. Its time range, 13:12:43 to 13:29:15 on 22 February 2025 (Figure 17), is about 16 minutes 32 seconds, and the integrity level reads System. The process folder names exported by MemProcFS confirm that PID 560 is `winlogon.exe` (Figure 18).

This matters because a program the user opened from Explorer should sit under `explorer.exe` with the user's own token. An instance with a SYSTEM token under winlogon is not a normal launch. The Prefetch file for `TOCTOU.EXE` was created in the same second, 13:12:43. Taken together, the exploit ran and succeeded at **13:12:43 UTC**, and that is the moment the attacker obtained system-level access.

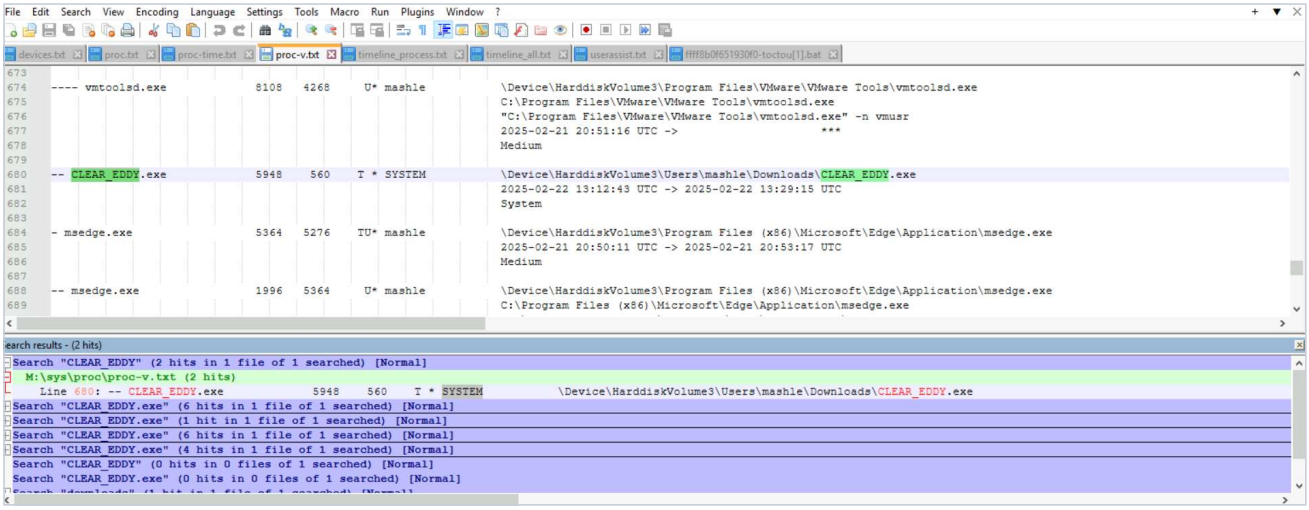


Figure 16. Search hits for CLEAR_EDDY in the verbose process list: one entry, owned by SYSTEM, image path in Downloads.

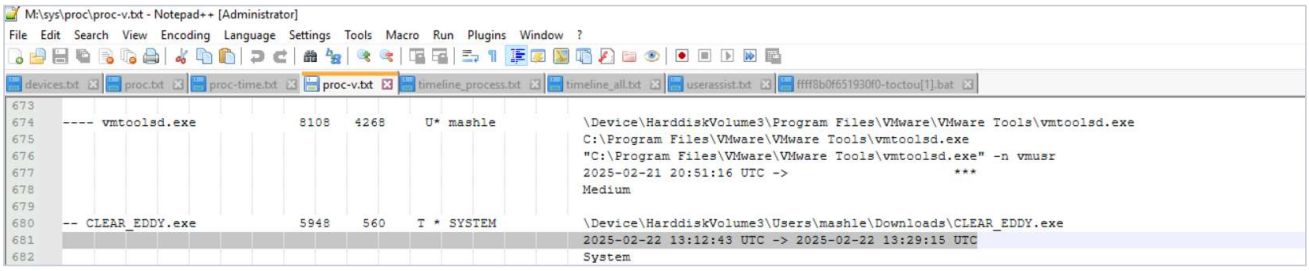


Figure 17. Process entry for CLEAR_EDDY.exe (PID 5948): start 13:12:43 and exit 13:29:15, integrity level System.

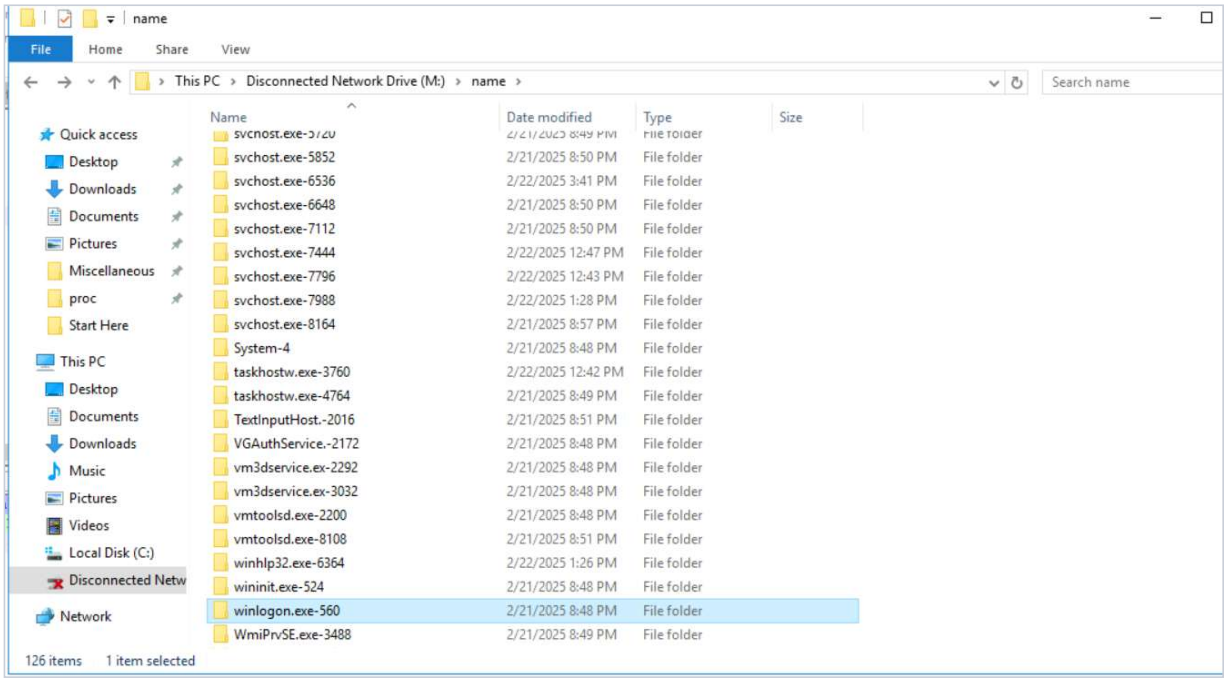


Figure 18. MemProcFS per-process folders: winlogon.exe-560, the parent of the SYSTEM-level CLEAR_EDDY.exe.

Phase 7 - Creation of a Rogue Administrator Account

With SYSTEM rights the attacker could change the account database. I opened the SAM hive recovered by MemProcFS in Registry Explorer and looked at the Users key (Figure 19). It lists two local accounts. `mashle` (RID 1000) was created on 21 February at 20:47:00 and belongs to Users. `gu3st` (RID 1001) was created on **22 February 2025 at 13:14:47 UTC** and belongs to **Administrators** and Users. It was created two minutes and four seconds after the SYSTEM process appeared, which looks like one of the attacker's first actions. The registry key for RID 0x3E9 was written again at 13:19:55, which suggests the account was touched again a few minutes later. This account is a backdoor that would survive removal of the original malware.

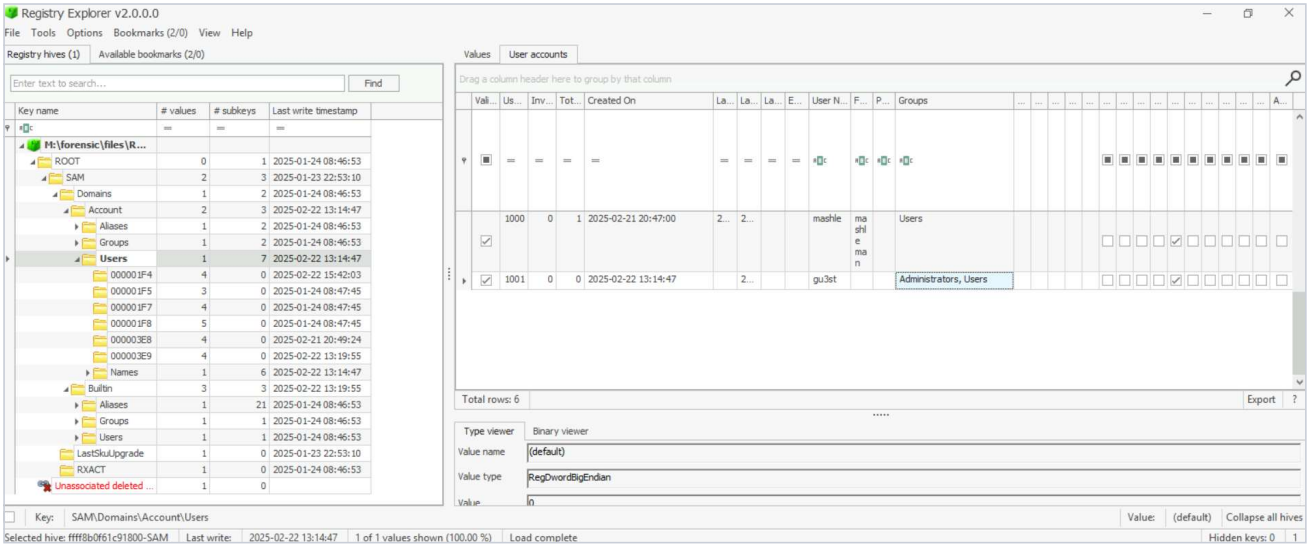


Figure 19. Registry Explorer, SAM\Domains\Account\Users: gu3st (RID 1001, Administrators and Users) created 2025-02-22 13:14:47.

Phase 8 - Analysis of the Batch Script

I read `toctou[1].bat` in Notepad++ (Figure 20). The recovered copy carried trailing NUL padding from the memory extraction, which I trimmed before reading it. The script is organized in labeled steps that each echo what they do and check the exit code, and it does three jobs: persistence, defense evasion and cleanup.

First it copies the downloaded payload into the system directory under a believable name: `copy "C:\Users\mashle\Downloads\CLEAR_EDDY.exe" "C:\Windows\System32\winhlp32.exe"` (Figure 21). Writing into System32 needs administrative rights, so this step could only happen after the escalation. The name imitates the legacy Windows Help executable. Next it sets the PowerShell execution policy to RemoteSigned for the current user and creates the `WindowsPowerShell` profile folder.

The persistence step follows (Figure 22): `echo C:\Windows\System32\winhlp32.exe > "%USERPROFILE%\Documents\WindowsPowerShell\Microsoft.PowerShell_profile.ps1"`. A PowerShell profile is a script that runs whenever PowerShell starts unless the `-NoProfile` flag is used, so every PowerShell session now tries to launch the masqueraded malware. The recovered profile file confirms it: its first line is the path of `winhlp32.exe`, followed by NUL padding from the extraction (Figure 23). The attacker therefore had two persistence paths, the rogue administrator account and the profile hook, and the second one also hides the malware behind a trusted-looking file name.

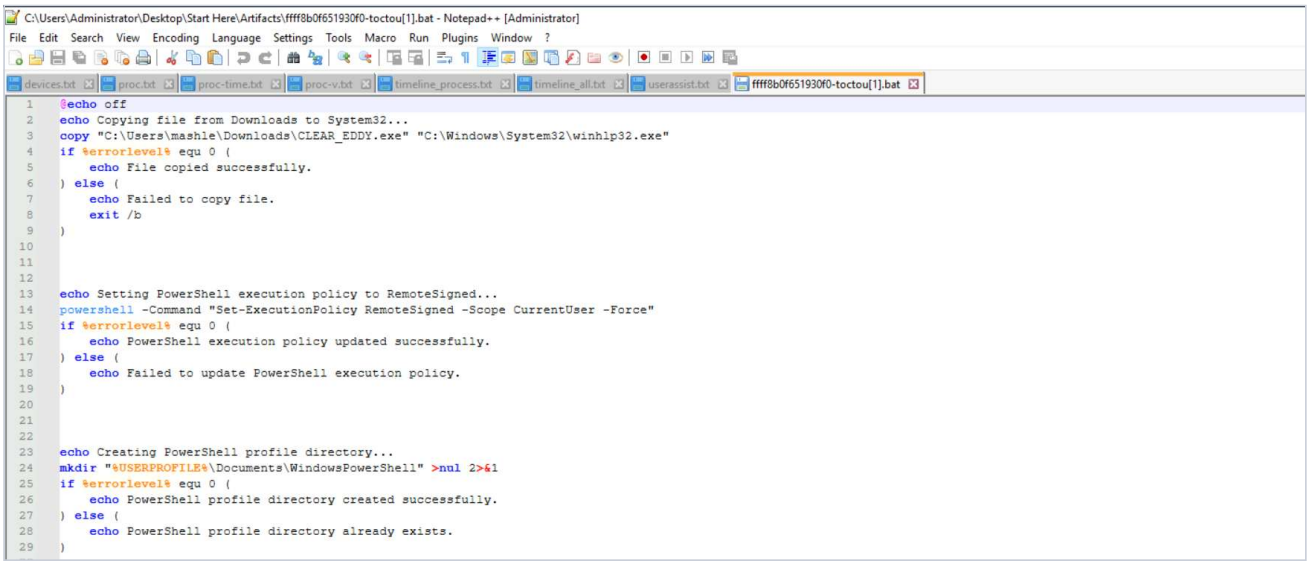


Figure 20. Start of toctou[1].bat: copy of CLEAR_EDDY.exe to System32, execution-policy change and profile-folder creation.

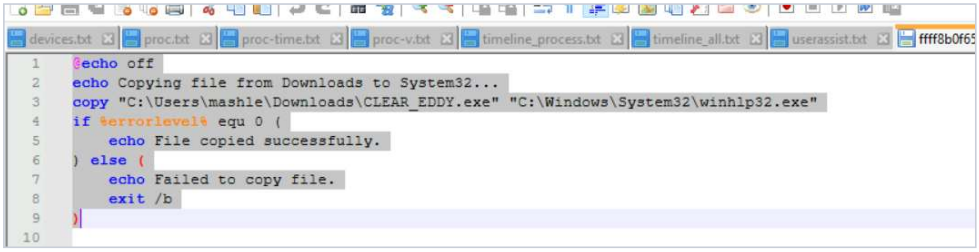


Figure 21. The copy block: CLEAR_EDDY.exe from Downloads is copied to C:\Windows\System32\winhlp32.exe.

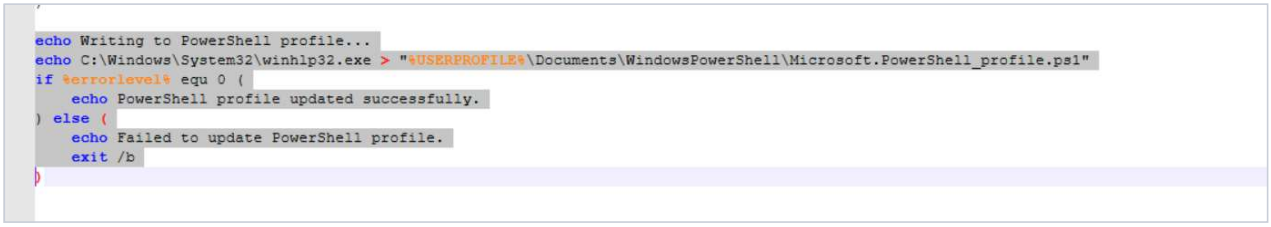


Figure 22. The persistence block: the path of winhlp32.exe is written into Microsoft.PowerShell_profile.ps1.

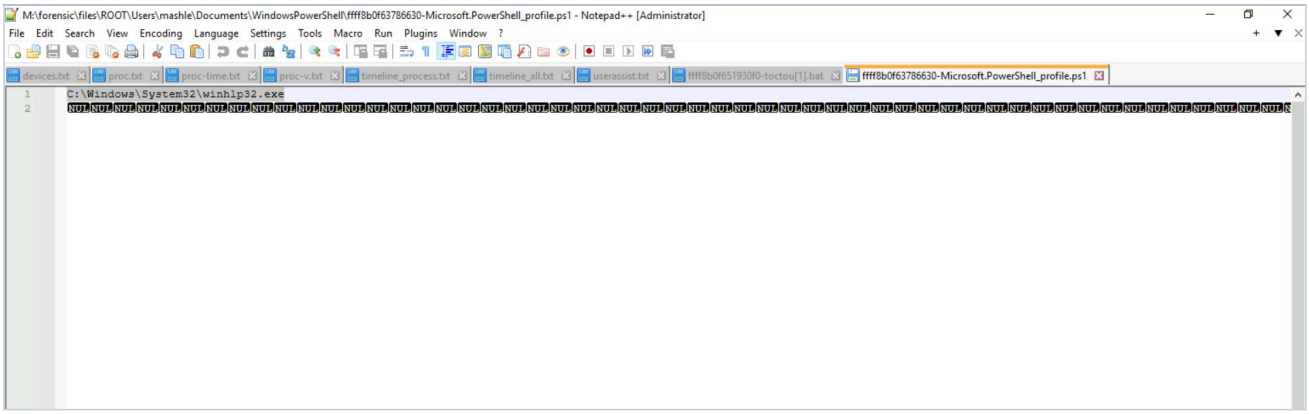


Figure 23. Recovered PowerShell profile from the user's Documents folder: line 1 is the path of winhlp32.exe; the NUL bytes are extraction padding.

Phase 9 - Mapping the Persistence and Masquerading Techniques

To classify the two techniques in the script I used the MITRE ATT&CK knowledge base. The profile hook is **Event Triggered Execution: PowerShell Profile (T1546.013)** (Figure 24), a sub-technique that the page files under both Persistence and Privilege Escalation and that needs only user or administrator rights on Windows.

For the System32 copy I first landed on the Masquerading entries of the Mobile matrix, T1655 and T1655.001 (Figures 25 and 26). They apply to Android and iOS only, so I discarded them for a Windows host. The Enterprise matrix entry **Masquerading: Rename Legitimate Utilities (T1036.003)** (Figure 27, named Rename System Utilities in earlier releases) fits: the malware was copied into the system directory under a system-looking file name so that detections keyed on file names or paths would miss it. The sidebar in the same figure lists Match Legitimate Resource Name or Location as a neighboring sub-technique, which describes a similar behavior.

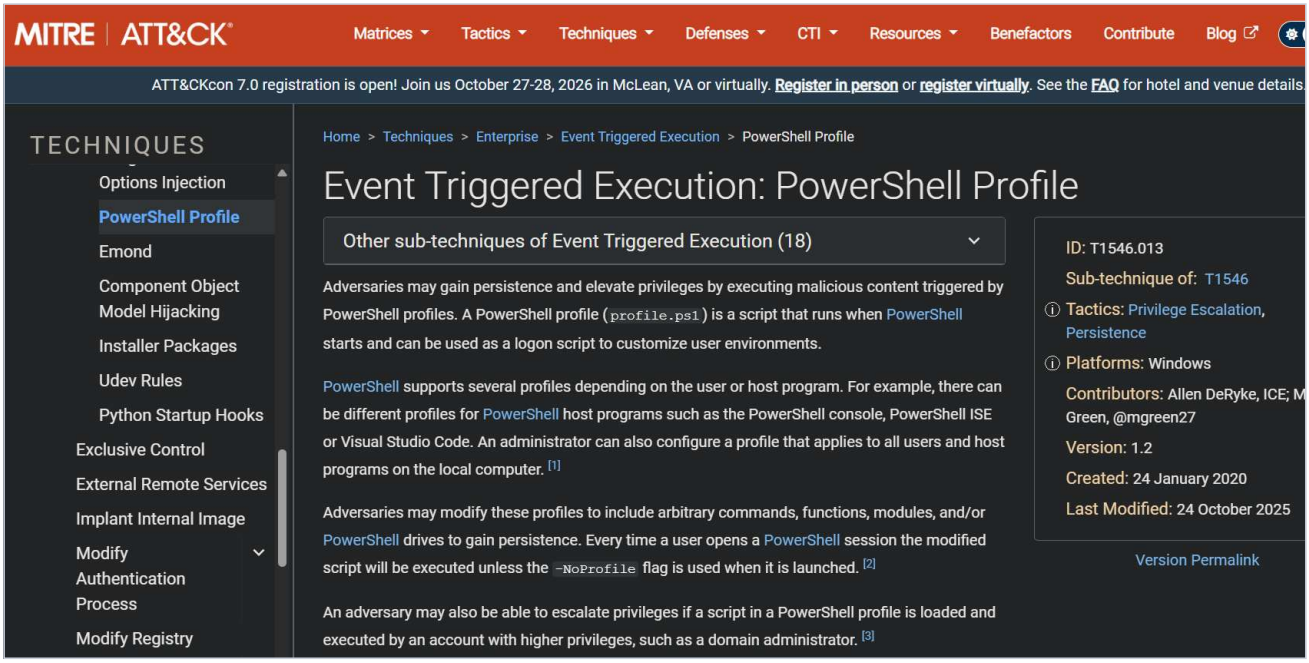


Figure 24. ATT&CK T1546.013, Event Triggered Execution: PowerShell Profile (Persistence, Privilege Escalation).

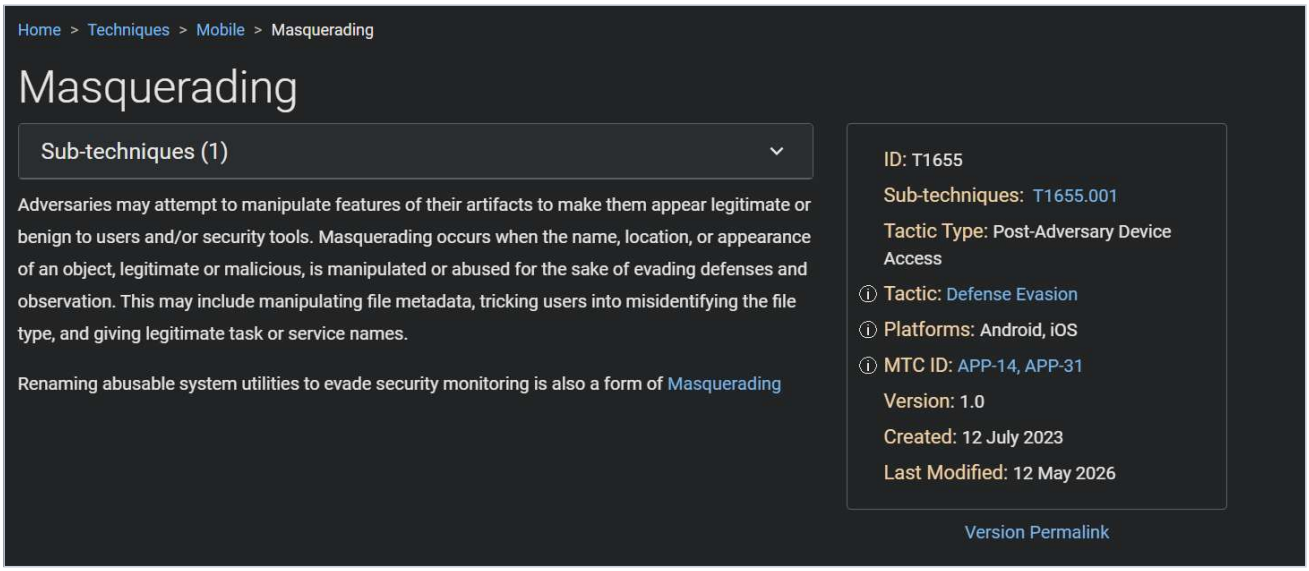


Figure 25. ATT&CK T1655, Masquerading, Mobile matrix (Android and iOS): not applicable to this Windows host.

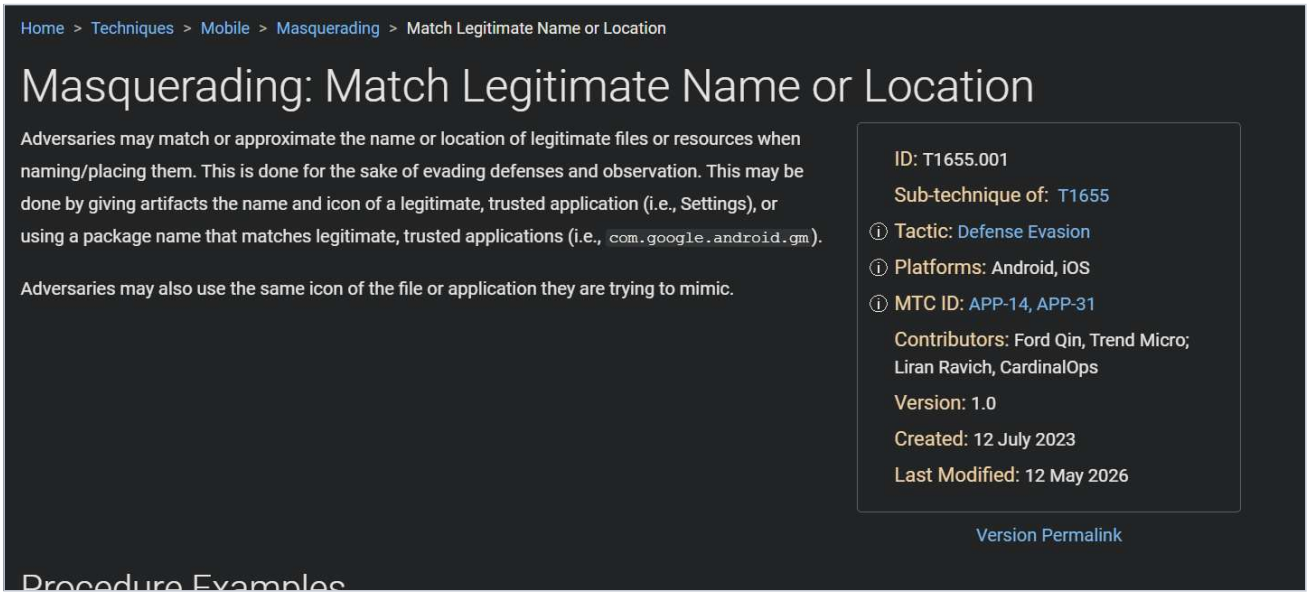


Figure 26. ATT&CK T1655.001, Match Legitimate Name or Location, Mobile matrix: also ruled out.

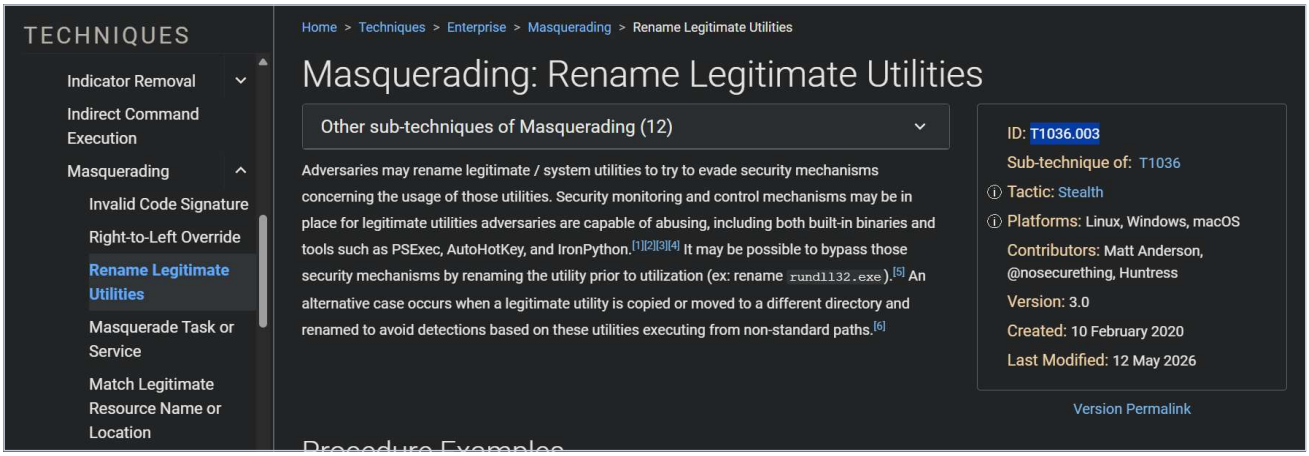


Figure 27. ATT&CK T1036.003, Masquerading: Rename Legitimate Utilities, Enterprise matrix (Linux, Windows, macOS).

Phase 10 - Command and Control Connection

To see whether the persistence worked, I examined the network view from MemProcFS (`netstat-v.txt`) and searched it for the masqueraded name (Figure 28). The hit is on line 37: PID 6364, `winhlp32.exe`, TCPv4, **ESTABLISHED**, from `192.168.1.27:57807` to **162.159.140.245:443**, with a creation time of 15:41:29 on 22 February. PID 6364 is the same SYSTEM process seen in Phase 1. The connection was opened about 35 seconds before DumpIt started, which means the implant was talking to its operator when the memory was captured, roughly two hours after the cleanup. The other established connections in the listing belong to Windows components such as the search app and svchost. The destination appears to be inside Cloudflare's address space, so the real backend may sit behind a proxy; a WHOIS lookup should confirm this before blocking anything by IP alone.

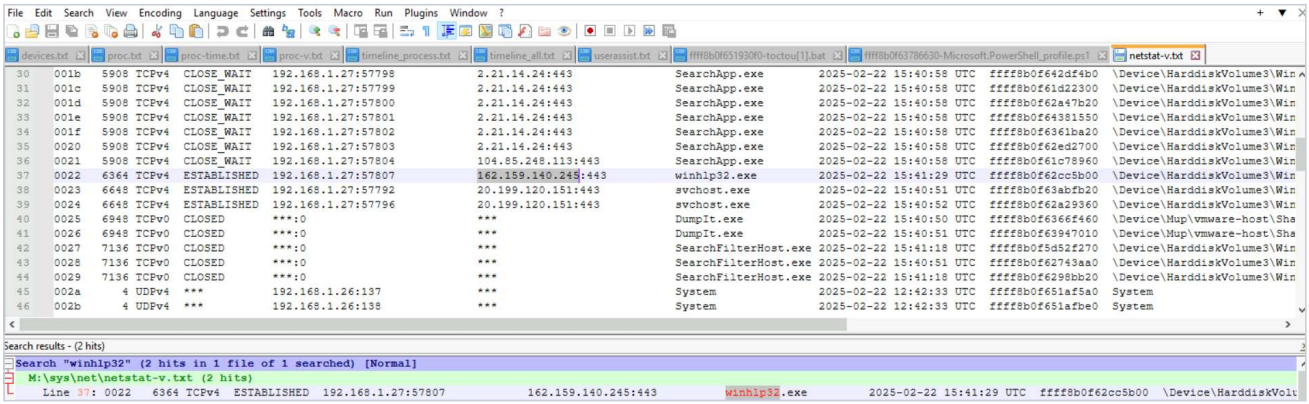


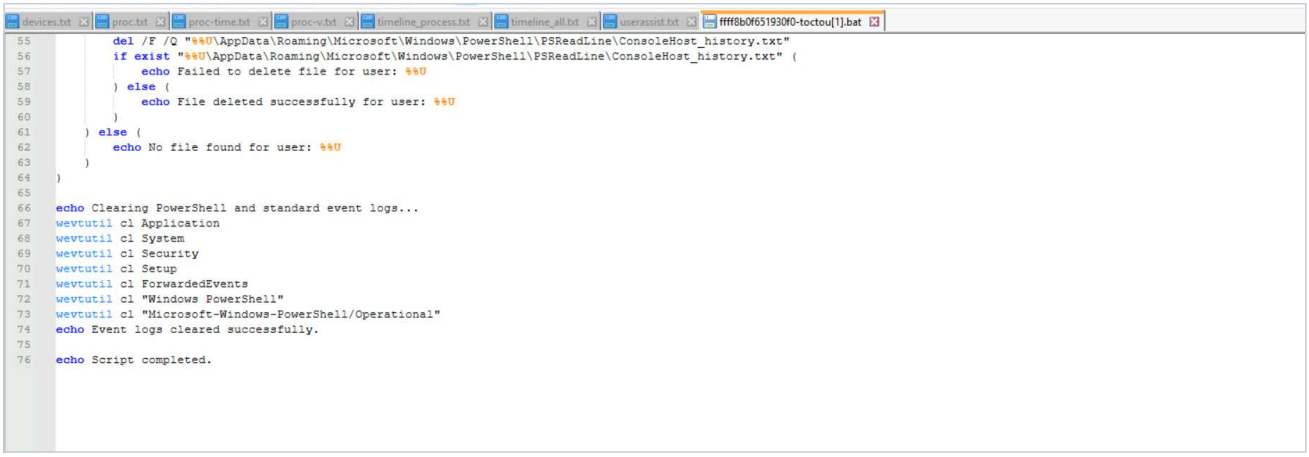
Figure 28. Network view searched for winhlp32: PID 6364 is ESTABLISHED to 162.159.140.245:443, created 2025-02-22 15:41:29.

Phase 11 - Cleanup and Anti-Forensics

The last part of the script deals with evidence (Figure 29). A loop walks every profile under `C:\Users` and deletes the PowerShell console history file `ConsoleHost_history.txt`, and the script then clears seven logs with `wevtutil cl`: Application, System, Security, Setup, ForwardedEvents, Windows PowerShell and Microsoft-Windows-PowerShell/Operational. Between them these logs hold most of the host-based record of process creation, logons and PowerShell commands.

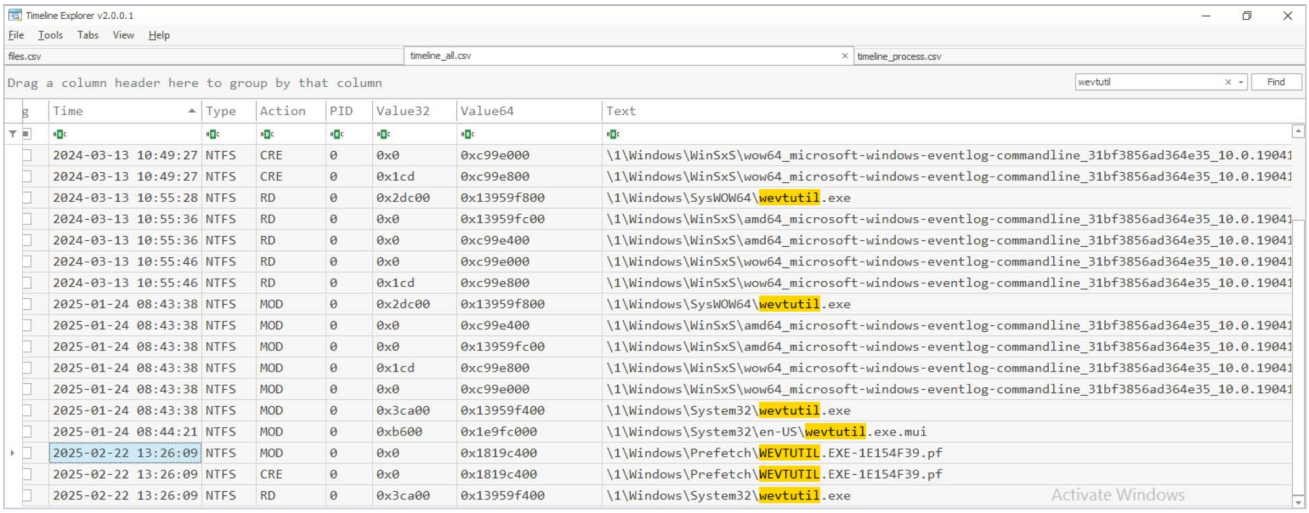
To date the cleanup I searched the timeline for wevtutil (Figure 30). The older rows are Windows servicing activity from March 2024 and January 2025. The relevant rows share one timestamp, **22 February 2025 at 13:26:09 UTC**: a Prefetch file for `WEVTUTIL.EXE` is created and modified, and `\Windows\System32\wevtutil.exe` is read. That is 13 minutes and 26 seconds after the SYSTEM process appeared, and one second after `winhlp32.exe` started at 13:26:08, so the masqueraded copy and the log clearing belong to the same burst of activity. The clearing worked well enough that the event logs could not be used here, and the attack had to be rebuilt from memory, Prefetch, UserAssist and the SAM hive, which the

script did not touch.



```
55          del /F /Q "%%U\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt"
56          if exist "%%U\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt" (
57              echo Failed to delete file for user: %%U
58          ) else (
59              echo File deleted successfully for user: %%U
60          )
61      ) else (
62          echo No file found for user: %%U
63      )
64  )
65
66  echo Clearing PowerShell and standard event logs...
67  wevtutil cl Application
68  wevtutil cl System
69  wevtutil cl Security
70  wevtutil cl Setup
71  wevtutil cl ForwardedEvents
72  wevtutil cl "Windows PowerShell"
73  wevtutil cl "Microsoft-Windows-PowerShell/Operational"
74  echo Event logs cleared successfully.
75
76  echo Script completed.
```

Figure 29. End of the script: deletion of PowerShell console history for each profile and wevtutil cl against seven logs.



Timeline Explorer v2.0.0.1

File Tools Tabs View Help

files.csv | timeline_all.csv | timeline_process.csv

Drag a column header here to group by that column

wevtutil | Find

Time	Type	Action	PID	Value32	Value64	Text
2024-03-13 10:49:27	NTFS	CRE	0	0x0	0xc99e000	\1\Windows\WinSxS\wow64_microsoft-windows-eventlog-commandline_31bf3856ad364e35_10.0.19041
2024-03-13 10:49:27	NTFS	CRE	0	0x1cd	0xc99e800	\1\Windows\WinSxS\wow64_microsoft-windows-eventlog-commandline_31bf3856ad364e35_10.0.19041
2024-03-13 10:55:28	NTFS	RD	0	0x2dc00	0x13959f800	\1\Windows\SysWOW64\wevtutil.exe
2024-03-13 10:55:36	NTFS	RD	0	0x0	0x13959fc00	\1\Windows\WinSxS\amd64_microsoft-windows-eventlog-commandline_31bf3856ad364e35_10.0.19041
2024-03-13 10:55:36	NTFS	RD	0	0x0	0xc99e400	\1\Windows\WinSxS\amd64_microsoft-windows-eventlog-commandline_31bf3856ad364e35_10.0.19041
2024-03-13 10:55:46	NTFS	RD	0	0x0	0xc99e000	\1\Windows\WinSxS\wow64_microsoft-windows-eventlog-commandline_31bf3856ad364e35_10.0.19041
2024-03-13 10:55:46	NTFS	RD	0	0x1cd	0xc99e800	\1\Windows\WinSxS\wow64_microsoft-windows-eventlog-commandline_31bf3856ad364e35_10.0.19041
2025-01-24 08:43:38	NTFS	MOD	0	0x2dc00	0x13959f800	\1\Windows\SysWOW64\wevtutil.exe
2025-01-24 08:43:38	NTFS	MOD	0	0x0	0xc99e400	\1\Windows\WinSxS\amd64_microsoft-windows-eventlog-commandline_31bf3856ad364e35_10.0.19041
2025-01-24 08:43:38	NTFS	MOD	0	0x0	0x13959fc00	\1\Windows\WinSxS\amd64_microsoft-windows-eventlog-commandline_31bf3856ad364e35_10.0.19041
2025-01-24 08:43:38	NTFS	MOD	0	0x1cd	0xc99e800	\1\Windows\WinSxS\wow64_microsoft-windows-eventlog-commandline_31bf3856ad364e35_10.0.19041
2025-01-24 08:43:38	NTFS	MOD	0	0x0	0xc99e000	\1\Windows\WinSxS\wow64_microsoft-windows-eventlog-commandline_31bf3856ad364e35_10.0.19041
2025-01-24 08:43:38	NTFS	MOD	0	0x3ca00	0x13959f400	\1\Windows\System32\wevtutil.exe
2025-01-24 08:44:21	NTFS	MOD	0	0xb600	0x1e9fc000	\1\Windows\System32\en-US\wevtutil.exe.mui
2025-02-22 13:26:09	NTFS	MOD	0	0x0	0x1819c400	\1\Windows\Prefetch\WEVTUTIL.EXE-1E154F39.pf
2025-02-22 13:26:09	NTFS	CRE	0	0x0	0x1819c400	\1\Windows\Prefetch\WEVTUTIL.EXE-1E154F39.pf
2025-02-22 13:26:09	NTFS	RD	0	0x3ca00	0x13959f400	\1\Windows\System32\wevtutil.exe

Activate Windows

Figure 30. Timeline search for wevtutil: Prefetch created and modified and wevtutil.exe read at 2025-02-22 13:26:09.

5. Attack Timeline

Date and time (UTC)	Event
2025-02-21 21:06:32	CLEAR_EDDY.zip download starts in Edge from bashupload.com
2025-02-22 12:43:13	User runs CLEAR_EDDY.exe from the Downloads folder (UserAssist, run count 1)
2025-02-22 12:43:23	Prefetch file for CLEAR_EDDY.EXE created
2025-02-22 13:10:47	certutil.exe Prefetch updated; toctou[1].bat appears in INetCache
2025-02-22 13:12:43	TOCTOU.EXE Prefetch created; CLEAR_EDDY.exe (PID 5948) starts as SYSTEM under winlogon.exe (CVE-2024-30088 exploited)
2025-02-22 13:12:53	CLEAR_EDDY.EXE Prefetch modified
2025-02-22 13:13:45	WHOAMI.EXE Prefetch updated
2025-02-22 13:14:47	Rogue administrator account gu3st (RID 1001) created
2025-02-22 13:26:08	winhlp32.exe (PID 6364) starts as SYSTEM
2025-02-22 13:26:09	wevtutil.exe executed; event logs cleared

Date and time (UTC)	Event
2025-02-22 13:29:15	SYSTEM-level CLEAR_EDDY.exe process exits
2025-02-22 15:41:29	winhlp32.exe has an established HTTPS connection to 162.159.140.245:443
2025-02-22 15:42:05	DumpIt.exe starts; memory image captured

Times come from the memory image artifacts. Prefetch times mark the last execution recorded for each program.

6. MITRE ATT&CK Mapping

Technique	ID	Observed activity
User Execution: Malicious File	T1204.002	User extracted and ran CLEAR_EDDY.exe from Downloads
Ingress Tool Transfer	T1105	certutil.exe used to download TOCTOU.exe and the batch script
Exploitation for Privilege Escalation	T1068	TOCTOU.exe exploited CVE-2024-30088 to obtain SYSTEM
Create Account: Local Account	T1136.001	Administrator account gu3st created
Event Triggered Execution: PowerShell Profile	T1546.013	Profile modified to launch winhlp32.exe whenever PowerShell starts
Masquerading: Rename Legitimate Utilities	T1036.003	Malware copied to System32 as winhlp32.exe
Command and Scripting Interpreter	T1059.001, T1059.003	Batch script and PowerShell used to configure persistence
Indicator Removal: Clear Windows Event Logs	T1070.001	wevtutil cl run against seven logs
Indicator Removal: Clear Command History	T1070.003	ConsoleHost_history.txt deleted for every profile
Application Layer Protocol: Web Protocols	T1071.001	HTTPS connection from winhlp32.exe to an external address, consistent with C2

7. Indicators of Compromise

Type	Value	Notes
File name	`CLEAR_EDDY.zip / CLEAR_EDDY.exe`	Initial payload, run from C:\Users\mashle\Downloads\
URL (defanged)	`hxxps://bashupload[.]com/prk5V/CLEAR_EDDY.zip`	Download source recorded in the Edge history
File name	`TOCTOU.exe (cached as TOCTOU[1].exe)`	CVE-2024-30088 exploit; INetCache\IE\FW98IITW\
MD5	`72FA3477AE8D9EBCD3266352E2C4D2D5`	TOCTOU.exe
SHA-256	`51c4e44fbe6764253887259923e8420ae70dffb3a19aed0d006567eecebbf3d9`	TOCTOU.exe
File name	`toctou[1].bat`	Persistence and cleanup script; INetCache\IE\CZFS7CMT\

Type	Value	Notes
File path	`C:\Windows\System32\winhlp32.exe`	Copy of CLEAR_EDDY.exe masquerading as a system component
File path	`C:\Users\mashle\Documents\WindowsPowerShell\Microsoft.PowerShell_profile.ps1`	Profile that launches winhlp32.exe
User account	`gu3st (RID 1001)`	Rogue local administrator, created 2025-02-22 13:14:47 UTC
IP address	`162.159.140.245:443`	HTTPS connection from winhlp32.exe (PID 6364)
Command	`certutil.exe download from a GitHub-hosted URL`	Retrieval of TOCTOU.exe
Command	`wevtutil cl (seven logs)`	Log clearing at 2025-02-22 13:26:09 UTC

8. Impact Assessment

The attacker went from a user-level program to SYSTEM on the workstation, the highest privilege on the machine. With that access they could read any data on the host, including the user's research files and browser data, and cached credentials. They also created a persistent administrator account, planted a second persistence mechanism and removed much of the host-based evidence. The live HTTPS connection at capture time shows the foothold was still in use. The host should be considered fully compromised, and any account that was used on it, or any system reachable from it, may be exposed.

9. Recommendations

- Isolate the workstation from the network now, and rebuild it from a known-good image rather than cleaning it, because the compromise was at SYSTEM level and the logs were wiped.
- Remove or disable the account `gu3st`, delete `C:\Windows\System32\winhlp32.exe` and the PowerShell profile content, and check every profile directory, including the all-users profiles, for similar hooks.
- Reset the password of mashle and of any account or service credential that was used on the host.
- Install the Microsoft update that fixes CVE-2024-30088 (released June 2024) or any later cumulative update on all Windows systems, and check that none are still unpatched.
- Verify who owns 162.159.140.245 before blocking it, and block at the proxy or DNS level as well as by IP. Search proxy and firewall logs for other hosts talking to it.
- Hunt across the estate for the file hashes, the file names, the bashupload.com URL, new accounts named gu3st and the winhlp32.exe file in System32.
- Alert on certutil.exe making outbound connections or using URL-download options, and restrict it with an application-control or firewall rule where it is not needed.
- Use application control such as AppLocker or WDAC to stop executables running from Downloads and user-writable folders, and warn users against running software from anonymous file-sharing links.
- Monitor for writes of new executables into System32 by non-installer processes and for modification of PowerShell profile files.
- Forward event logs to a central collector so that wevtutil cl cannot erase the only copy, and alert on log-cleared events (Security 1102 and System 104).

10. Conclusion

The memory image shows that the workstation was compromised through a user-run download and a local privilege-escalation exploit for CVE-2024-30088. Within about half an hour of the first run, the attacker had SYSTEM access, a rogue administrator account and a pair of persistence mechanisms, and about 43 minutes after the first run they cleared the event logs. The masqueraded malware was still communicating with an external address when the memory was captured. Patching, credential rotation, a rebuild of the host and the detections listed above are needed to remediate the incident and catch similar activity earlier.